Lab 6: Code and Branch Coverage Analysis

Objective

- Understand the difference between **code coverage** and **branch coverage**.
- Use coverage to **measure**, **analyse**, and **improve** test coverage.
- Interpret coverage reports and identify **untested paths** in code.
- Practice writing test cases that cover **edge cases and decision branches**.

Duration:

60 minutes

Software Requirements

- Python 3.x
- coverage (install via pip)
- Any code editor (VS Code / PyCharm / Sublime / etc.)
- pytest (install via pip)

💡 Coverage can be used standalone or with test frameworks.

Learning Outcomes

- **Understand Code Coverage Concepts**: You will differentiate between line coverage, branch coverage, path coverage, and function coverage to measure test completeness.
- **Apply Coverage Analysis Tools**: You will use Python's coverage library to generate reports, interpret coverage metrics, and identify untested code paths.
- **Design Comprehensive Test Cases**: You will write targeted test cases to achieve minimum 90% to 100% coverage by addressing edge cases and decision branches.

Introduction

What is Code Coverage?

Code coverage measures how much of your source code is **executed during testing**.

Types of coverage:

Type	Meaning
Line Coverage	How many lines of code were executed
Branch Coverage	How many branches (e.g., if, else) were followed
Path Coverage	How many unique paths through code were tested
Function Coverage	How many functions were invoked during tests

Why It Matters:

- High coverage improves **confidence** in correctness.
- Helps catch **untested paths** and **dead code**.
- Encourages **test completeness**, especially in critical systems

Example:

1. Example Code and Tests

calculator.py (The code to test)

```
def calculate_discount(price, is_member):
      discount = 0
      if is_member:
            discount = 0.10
      if price > 100:
            discount += 0.05
      return price * (1 - discount)
```

test_calculator.py (The test cases)

```
import unittest
from calculator import calculate_discount

class TestCalculator(unittest.TestCase):
      def test_member_expensive_item(self):
            # Tests when is_member is True AND price > 100
            result = calculate_discount(150, True)
            self.assertEqual(result, 150 * 0.85) # 15% total
      discount
if __name__ == '__main__':
      unittest.main()
```

2. Commands to Run Code Coverage

Run tests and gather line coverage:

```
coverage run -m pytest test_calculator.py
```

View the line coverage report:

```
coverage report -m
```

Note: The -m flag shows exactly which lines are missing coverage.

Run tests and gather branch coverage:

```
coverage run --branch -m pytest test_calculator.py
```

View the branch coverage report:

```
coverage report -m
```

3. Line Coverage vs. Branch Coverage
The test case above perfectly illustrates the difference between line and branch coverage.

Line Coverage (100% in this example): Line coverage simply asks: "Did the execution pass through this line of code at least once?" When we pass 150 and True into our test, is_member is true (triggering line 4), and price > 100 is true (triggering line 7). Because every single line in calculator.py executes at least once, your line coverage will report 100%.

Branch Coverage (Partially covered in this example): Branch coverage asks: "Did every control structure (if, while, etc.) evaluate to both True and False?" Even though our line coverage is 100%, our branch coverage will report missing branches. Why?

We tested what happens when is_member is True, but we never tested the branch where is_member is False.
We tested what happens when price > 100, but we never tested the branch where price <= 100.

Line coverage can give a false sense of security by ignoring alternate logical paths.
Branch coverage forces you to test all possible pathways and edge cases through your code.
To get 100% branch coverage on the code above, you would need to add test cases where is_member is False, and where the price is 100 or less.

Steps:

1. **Install dependencies:**
   pip install coverage pytest

2. Access the **individual repository assigned to you**. Clone and work locally.

3. Each repo will have two python scripts. One is the application and the other is the test case.

4. Run the code and the branch coverage for the initial test case.

5. Write more test cases in the test file to achieve at least 90% code coverage. Re-run steps 2-4 and provide similar screenshots as above for the results. Give screenshots ONLY for results after achieving at least 90% code coverage.

6. Commit the test case and screenshot to your repo. (code coverage).

7. Write more test cases in the test file to achieve at least 90% branch coverage. Re-run steps 2-4 and provide similar screenshots as above for the results. Give screenshots ONLY for results after achieving at least 90% branch coverage.

8. Commit the test case and screenshot to your repo. (branch coverage)

Deliverables:

- **Coverage (Code and Branch) analysis (Terminal Output Screenshots)**
- **CodeCoverageTest_xxxx_application.py (test cases for code coverage)**
- **BranchCoverageTest_xxxx_application.py (test cases for branch coverage)**
- **Minimum 2 commits. One for updated code coverage test case, and other for branch coverage test case.**

Evaluation Rubric

Criteria	**Marks**
Code Coverage analysis & report	3
Branch Coverage analysis & report	3
Commits and Testcases (>90% Coverage)	6
Total	**10**